Литералы и магические числа

В языке C++ есть два вида констант: литеральные и символьные. На этом уроке мы рассмотрим литеральные константы.

Оглавление:

1. **Литеральные константы**
2. **Литералы в восьмеричной и шестнадцатеричной системах счисления**
3. **Бинарные литералы и разделитель цифр в C++14**
4. **Магические числа. Что с ними не так?**

Литеральные константы

Литеральные константы (или просто ***«литералы»***) — это значения, которые вставляются непосредственно в код. Поскольку они являются константами, то их значения изменить нельзя. Например:

```
bool myNameIsAlex = true; // true - это литеральная константа типа bool
int x = 5; // 5 - это литеральная константа типа int
int y = 2 * 3; // 2 и 3 - это литеральные константы типа int
```

С литералами типов **bool** и **int** всё понятно, а вот для литералов **типа с плавающей точкой** есть два способа определения:

```
double pi = 3.14159; // 3.14159 - это литерал типа double
double avogadro = 6.02e23; // число avogadro = 6.02 x 10^23
```

Во втором способе определения, число после экспонента может быть и отрицательным:

```
double electron = 1.6e-19; // заряд электрона = 1.6 x 10^-19
```

Числовые литералы могут иметь суффиксы, которые определяют их типы. Эти суффиксы не являются обязательными, так как компилятор понимает из контекста, константу какого типа данных вы хотите использовать.

Тип данных	Суффикс	Значение
int	u или U	unsigned int
int	l или L	long
int	ul, uL, Ul, UL, lu, lU, Lu или LU	unsigned long
int	ll или LL	long long
int	ull, uLL, Ull, ULL, llu, llU, LLu или LLU	unsigned long long
double	f или F	float
double	l или L	long double

Суффиксы есть даже для целочисленных типов (но они почти не используются):

```
unsigned int nValue = 5u; // тип int unsigned
long nValue2 = 5L; // тип long
```

По умолчанию литеральные константы типа с плавающей точкой являются типа double. Для конвертации литеральных констант в тип float можно использовать суффикс `f` или `F`:

```
float fValue = 5.0f; // тип float
double d = 6.02e23; // тип double (по умолчанию)
```

Язык C++ также поддерживает литералы типов string и **char**:

```
char c = 'A'; // 'A' - это литерал типа char
std::cout << "Hello, world!"; // "Hello, world!" - это литерал строки C-style
std::cout << "Hello,\" world!"; // C++ понимает escape-последовательные литералы типа string
```

Литералы хорошо использовать в коде до тех пор, пока их значения понятны и однозначны. Это выполнение операций присваивания, математических операций или вывода текста в консоль.

Литералы в восьмеричной и шестнадцатеричной системах счисления

В повседневной жизни мы используем **десятичную систему счисления**, которая состоит из десяти цифр: 0, 1, 2, 3, 4, 5, 6, 7, 8 и 9. По умолчанию язык C++ использует десятичную систему счисления для чисел в программах:

```
int x = 12; // предполагается, что 12 является числом десятичной системы счисления
```

В **двоичной (бинарной) системе счисления** всего 2 цифры: 0 и 1. Значения: 0, 1, 10, 11, 100, 101, 110, 111 и т.д.

Есть еще две другие системы счисления: восьмеричная и шестнадцатеричная.

Восьмеричная система счисления состоит из 8 цифр: 0, 1, 2, 3, 4, 5, 6 и 7. Значения: 0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12 и т.д.

> ***Примечание:*** В восьмеричной системе счисления нет цифр 8 и 9, так что сразу перескакиваем от 7 к 10.

Десятичная система счисления	0	1	2	3	4	5	6	7	8	9	10	11
Восьмеричная система счисления	0	1	2	3	4	5	6	7	10	11	12	13

Для использования литерала из восьмеричной системы счисления, используйте префикс `0` (ноль):

```
#include <iostream>

int main()
{
    int x = 012; // 0 перед значением означает, что это восьмеричный литерал
    std::cout << x;
    return 0;
}
```

Результат выполнения программы:

```
10
```

Почему 10 вместо 12? Потому что **std::cout** выводит числа в десятичной системе счисления, а 12 в восьмеричной системе = 10 в десятичной.

Восьмеричная система счисления используется крайне редко.

Шестнадцатеричная система счисления состоит из 16 символов: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Десятичная система	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Шестнадцатеричная система	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11

Для использования литерала из шестнадцатеричной системы счисления, используйте префикс `0x`:

```
#include <iostream>

int main()
{
    int x = 0xf; // 0x перед значением означает, что это шестнадцатеричный литерал
    std::cout << x;
    return 0;
}
```

Результат выполнения программы:

```
15
```

Поскольку в этой системе 16 символов, то одна шестнадцатеричная цифра занимает 4 бита. Следовательно, две шестнадцатеричные цифры занимают 1 байт.

Рассмотрим 32-битное целое число из двоичной системы счисления: `0011 1010 0111 1111 1001 1000 0010 0110`. Из-за длины и повторения цифр его сложно прочесть. В шестнадцатеричной системе счисления это же значение будет выглядеть следующим образом: `3A7F 9826`. Такой удобный/сжатый формат записи является преимуществом шестнадцатеричной системы счисления, поэтому шестнадцатеричные значения часто используются для представления адресов или необработанных значений в памяти.

До **C++14** использовать литерал из двоичной системы счисления было невозможно. Тем не менее, шестнадцатеричная система счисления может нам в этом помочь:

```
#include <iostream>

int main()
{
    int bin(0);
    bin = 0x81; // присваиваем переменной бинарный литерал 0000 0001
    bin = 0x02; // присваиваем переменной бинарный литерал 0000 0010
    bin = 0x04; // присваиваем переменной бинарный литерал 0000 0100
    bin = 0x08; // присваиваем переменной бинарный литерал 0000 1000
    bin = 0x10; // присваиваем переменной бинарный литерал 0001 0000
    bin = 0x20; // присваиваем переменной бинарный литерал 0010 0000
    bin = 0x40; // присваиваем переменной бинарный литерал 0100 0000
    bin = 0x80; // присваиваем переменной бинарный литерал 1000 0000
    bin = 0xff; // присваиваем переменной бинарный литерал 1111 1111
    bin = 0x83; // присваиваем переменной бинарный литерал 1011 0011
    bin = 0xf770; // присваиваем переменной бинарный литерал 1111 0111 0111 0000

    return 0;
}
```

Бинарные литералы и разделитель цифр в C++14

В C++14 мы можем использовать бинарные (двоичные) литералы, добавляя префикс `0b`:

```
#include <iostream>

int main()
{
    int bin(0);
    bin = 0b1; // присваиваем переменной бинарный литерал 0000 0001
    bin = 0b11; // присваиваем переменной бинарный литерал 0000 0011
    bin = 0b1010; // присваиваем переменной бинарный литерал 0000 1010
    bin = 0b11110000; // присваиваем переменной бинарный литерал 1111 0000

    return 0;
}
```

Поскольку длинные литералы читать трудно, то в C++14 добавили возможность использовать одинарную кавычку `'` в качестве разделителя цифр:

```
#include <iostream>

int main()
{
    int bin = 0b1011'0010; // присваиваем переменной бинарный литерал 1011 0010
    long value = 2'532'673'462; // для намного проще читать, нежели 2532673462

    return 0;
}
```

Если ваш компилятор не поддерживает C++14, то использовать бинарные литералы и разделитель цифр вы не сможете — компилятор выдаст ошибку.

Магические числа. Что с ними не так?

Рассмотрим следующий фрагмент кода:

```
int maxStudents = numClassrooms * 30;
```

В вышеприведенном примере число 30 является магическим числом. **Магическое число** — это хорошо закодированный литерал (обычно, число) в строке кода, который не имеет никакого контекста. Что это за 30, что оно означает/обозначает? Хотя из вышеприведенного примера можно догадаться, что число 30 обозначает максимальное количество учеников, находящихся в одном кабинете — в большинстве случаев, это не будет столь очевидным и понятным. В более сложных программах контекст подобных чисел разгадать намного сложнее (если только не будет соответствующих **комментариев**).

Использование магических чисел является плохой практикой, так как в дополнение к тому, что они не предоставляют никакого контекста (для чего и почему используются), они также могут создавать проблемы, если их значения необходимо будет изменить. Предположим, что школа закупила новые парты. Эта покупка, соответственно, увеличила максимально возможное количество учеников, находящихся в одном кабинете, с 30 до 36 — это нужно будет продумать и отобразить в нашей программе.

Рассмотрим следующий фрагмент кода:

```
int maxStudents = numClassrooms * 30;
setMax(30);
```

Чтобы обновить число учеников в кабинете, нам нужно изменить значение константы с 30 на 36. Но что делать с вызовом функции setMax(30)? **Аргумент** 30 и константа 30 в коде, приведенном выше, являются одним и тем же, верно? Если да, то нам нужно будет обновить это значение. Если нет, то нам не следует вообще трогать этот вызов функции. Если же проводить автоматический глобальный поиск и замену числа 30, то можно ненароком изменить и аргумент функции setMax(), в то время, когда его вообще не следовало бы трогать. Поэтому вам придется просмотреть весь код «вручную», в поисках числа 30, а затем в каждом конкретном случае определить — изменить ли 30 на 36 или нет. Это может занять очень много времени, кроме того, вероятность возникновения новых ошибок повышается в разы.

К счастью, есть лучший вариант — использовать символьные константы. О них мы поговорим на следующем уроке.

Правило: Старайтесь свести к минимуму использование магических чисел в ваших программах.